Kịch bản nói — Bảo vệ ĐATN PM QLKT

> *File này **chỉ chứa lời văn** em đọc khi thuyết trình — bám sát thứ tự 19 slide trong `defense.marp.md`. Không đọc nguyên văn bullet trên slide.*

Hướng dẫn dùng

Mục	Chi tiết
Tổng thời lượng	13–14 phút (chừa ~10–15 phút Q&A)
Nhịp đọc	130–140 từ/phút (~2,2 từ/giây). Chậm rãi, ngắt câu rõ
Xưng hô	"Em" / "Hội đồng" hoặc "thầy/cô". Trang trọng, không "tôi", không "các bạn"
Số liệu	Đọc dạng chữ ("hai mươi ba bảng", "bảy mươi tư file"), không đọc số kiểu kỹ thuật

Cấu trúc mỗi slide

- **[Mở slide]** — 1 câu giới thiệu, để hội đồng kịp nhìn slide.
- **[Lời văn]** — đoạn văn nói chính (prose, không bullet). Trong đó:
  - `[ngừng]` — dừng ~1 giây để hội đồng tiếp thu
  - `[nhấn]` — đọc to + chậm hơn câu đó (điểm nhớ)
  - `[nhìn slide]` — quay xuống chỉ vào slide rồi quay lại nhìn hội đồng
- **[Chuyển]** — câu nối sang slide kế (luôn nói TRƯỚC khi click).

Cách đọc thuật ngữ tiếng Anh

Viết trên slide	Đọc khi nói
`Next.js`	"Nếch chấm jés" hoặc "Nếch jés"
`Node.js` / `node-cron`	"Nốt chấm jés" / "nốt-crón"
`Express`	"Ếch-prét" (giữ nguyên)
`PostgreSQL`	"Pót-grét" hoặc "Pót-grét quây-eo"
`Prisma`	"Prít-mà"
`TypeScript`	"Tai-scríp"
`Socket.IO`	"Sốc-két ai-ô"
`JWT`	"Jây Đáp-bờ-liu Ti" hoặc "Jót" — chọn 1 và giữ nhất quán
`PM2`	"Pi-em hai"
`ORM`	"Ô-e-em"
`BKBQP / CSTDTQ / BKTTCP`	Đọc nguyên: "Bằng khen Bộ Quốc phòng" / "Chiến sĩ Thi đua Toàn quốc" / "Bằng khen Thủ tướng Chính phủ" — tuyệt đối **không** đọc viết tắt

`HCBVTQ / HCCSVV / HCQKQT / KNC`	Đọc đầy đủ: "Huân chương Bảo vệ Tổ quốc" / "Huy chương Chiến sĩ Vẻ vang" / "Huân chương Quân kỳ Quyết thắng" / "Kỷ niệm chương Vì sự nghiệp xây dựng Quân đội"
`CSTDCS`	"Chiến sĩ Thi đua Cơ sở"

Slide 1 — Trang bìa (40 giây)

[Mở slide]: Trang tiêu đề.

[Lời văn]:

> *(Đứng thẳng, hai tay khoanh trước, cúi nhẹ chào hội đồng — khoảng 1 giây)*
>
> *Em kính chào các thầy, cô trong Hội đồng. [ngừng]*
>
> *Em xin tự giới thiệu, em là [Họ tên], mã số sinh viên [MSSV], lớp [tên lớp], chuyên ngành [chuyên ngành] — Trường Công nghệ Thông tin và Truyền thông, Đại học Bách khoa Hà Nội.*
>
> *Hôm nay em xin được trình bày đồ án tốt nghiệp với đề tài **[nhấn] "Phần mềm Quản lý Khen thưởng"**, được thực hiện dưới sự hướng dẫn tận tình của [Học hàm. Tên giảng viên]. Trước khi bắt đầu, em xin gửi lời cảm ơn chân thành đến thầy/cô đã hướng dẫn em trong suốt quá trình thực hiện đồ án.*
>
> *Em xin phép Hội đồng được bắt đầu phần trình bày ạ.*

[Chuyển]: *Trước khi vào nội dung chi tiết, em xin trình bày mục lục bài thuyết trình.*

Slide 2 — Nội dung trình bày (20 giây)

[Mở slide]: Mục lục 4 phần.

[Lời văn]:

> *Bài trình bày của em chia làm bốn phần. Phần một là mở đầu, gồm mục tiêu, phân tích bài toán và logic chuỗi danh hiệu — đây là nội dung nghiệp vụ phức tạp nhất. Phần hai trình bày thiết kế hệ thống. Phần ba là bốn nhóm tính năng nổi bật. Phần bốn là kết quả kiểm thử, triển khai và kết luận.*

[Chuyển]: *Em xin bắt đầu với mục tiêu của đồ án.*

Slide 3 — Mục tiêu của đồ án (55 giây)

[Mở slide]: 2 cột — Bối cảnh (trái) + Mục tiêu cụ thể (phải).

[Lời văn]:

> *Hiện nay, công tác quản lý khen thưởng tại các đơn vị quân đội chủ yếu dựa vào file Excel và hồ sơ giấy. Khi cán bộ cần xét những danh hiệu cấp cao như Bằng khen Bộ Quốc phòng, Chiến sĩ Thi đua Toàn quốc hay Bằng khen Thủ tướng Chính phủ, họ phải tra cứu lịch sử nhiều năm và áp dụng các quy định phức tạp như chuỗi danh hiệu, cửa sổ trượt 3 năm, 7 năm — rất dễ sai sót và mất thời gian.*
>
> *Đồ án của em đặt ra bốn mục tiêu cụ thể: thứ nhất, xây dựng phần mềm quản lý đầy đủ năm nhóm khen thưởng theo quy định; thứ hai, tự động hoá việc xét đủ điều kiện theo các luật hiện hành; thứ ba, hỗ trợ quy trình đề xuất, duyệt và ra quyết định có truy vết đầy đủ; và thứ tư là yêu cầu vận hành được trên mạng nội bộ, không phụ thuộc Internet — đây là yêu cầu bảo mật đặc thù của đơn vị quân đội.*

[Chuyển]: *Để hiểu rõ hơn vì sao bài toán này khó, em xin phân tích sáu nhóm thách thức ở slide tiếp theo.*

Slide 4 — Phân tích bài toán (50 giây)

[Mở slide]: 6 ô — Nghiệp vụ phức tạp / Logic chuỗi / Phân quyền / Real-time / Audit / Offline.

[Lời văn]:

> *Bài toán có sáu thách thức chính. Thứ nhất là nghiệp vụ phức tạp — năm nhóm khen thưởng, mỗi nhóm có quy định xét duyệt riêng. Thứ hai là logic chuỗi danh hiệu với cửa sổ trượt 3 và 7 năm cùng ràng buộc lifetime — phần này em sẽ trình bày kỹ ở slide kế tiếp vì đây là nội dung cốt lõi nhất. Thứ ba là phân quyền bốn cấp với phạm vi dữ liệu khác nhau theo cây đơn vị. Thứ tư là yêu cầu thông báo thời gian thực mỗi khi đề xuất chuyển trạng thái. Thứ năm là yêu cầu truy vết toàn diện cho công tác kiểm tra nội bộ. Và cuối cùng là triển khai hoàn toàn offline trong mạng nội bộ.*

[Chuyển]: *Em xin đi vào nội dung trọng tâm — hệ thống khen thưởng và logic chuỗi.*

Slide 5 — Hệ thống khen thưởng 5 nhóm UC5–UC9 (55 giây)

[Mở slide]: Bảng 5 dòng — UC5 đến UC9.

[Lời văn]:

> *Hệ thống quản lý năm nhóm khen thưởng. Nhóm UC5 là khen thưởng hằng năm, gồm cá nhân với chuỗi danh hiệu Chiến sĩ Tiên tiến, Chiến sĩ Thi đua Cơ sở, Bằng khen Bộ Quốc phòng, Chiến sĩ Thi đua Toàn quốc, Bằng khen Thủ tướng; và đơn vị với Đơn vị Quyết thắng và các bằng khen tương ứng. Nhóm UC6 là khen thưởng theo niên hạn — Huy chương Chiến sĩ Vẻ vang ba hạng, Huân chương Quân kỳ Quyết thắng và Kỷ niệm chương. Nhóm UC7 là Huân chương Bảo vệ Tổ quốc cống hiến, xét theo 120 tháng cộng dồn hệ số chức vụ. Nhóm UC8 là thành tích nghiên cứu khoa học. Và UC9 là khen thưởng đột xuất theo sự kiện.*
>
> *Mỗi nhóm này được hiện thực thành các loại đề xuất riêng và dispatch qua Strategy pattern ở backend — phần em sẽ giải thích sau.*

[Chuyển]: *Trong các nhóm này, phức tạp nhất là chuỗi danh hiệu hằng năm — em xin trình bày ở slide tiếp.*

Slide 6 — Chuỗi danh hiệu hằng năm — Logic cốt lõi (90 giây) ⭐⭐⭐

[Mở slide]: 2 cột — Chuỗi cá nhân (trái) + Quy tắc đặc biệt (phải).

[Lời văn]:

> *[nhấn] Đây là phần em đầu tư nhiều thời gian nhất trong đồ án — và cũng là phần khó nhất về mặt nghiệp vụ. [ngừng]*
>
> *[nhìn slide] Chuỗi cá nhân gồm **ba mốc**. Một quân nhân đạt **Chiến sĩ Thi đua Cơ sở hai năm liên tiếp** thì đủ điều kiện đề nghị **Bằng khen Bộ Quốc phòng**. Tiếp theo, có Bằng khen Bộ Quốc phòng và đủ ba năm chuỗi thì đề nghị **Chiến sĩ Thi đua Toàn quốc**. Và cao nhất, sau bảy năm với ba Bằng khen Bộ Quốc phòng, hai Chiến sĩ Thi đua Toàn quốc cộng với nghiên cứu khoa học hằng năm — thì đủ điều kiện **Bằng khen Thủ tướng Chính phủ**. [ngừng]*
>
> *Có **ba quy tắc đặc biệt** em đã hiện thực và xin được nhấn mạnh.*

> ***Thứ nhất là cửa sổ trượt.*** *[ngừng] Khi xét Chiến sĩ Thi đua Toàn quốc năm 2026, hệ thống chỉ đếm Bằng khen Bộ Quốc phòng trong* ***ba năm gần nhất*** *— tức là 2024 đến 2026. Các bằng khen của chu kỳ trước tự rơi ra khỏi cửa sổ. [nhấn] Đây là điểm rất dễ tính sai khi làm thủ công bằng Excel.*
>
> ***Thứ hai là lifetime.*** *[ngừng] Bằng khen Thủ tướng Chính phủ cá nhân chỉ được nhận* ***một lần duy nhất*** *trong đời. Sau khi nhận, hệ thống chặn mọi đề xuất tiếp theo và hiển thị thông báo rõ ràng — vì luật hiện hành chưa quy định danh hiệu cao hơn cho cá nhân.*
>
> ***Thứ ba là lỡ đợt.*** *[ngừng] Nếu đến mốc đủ điều kiện mà đơn vị không đề xuất, chu kỳ* ***vẫn tiếp tục đếm*** *— đến chu kỳ sau lại được xét, không yêu cầu đứt chuỗi Chiến sĩ Thi đua Cơ sở. [ngừng]*
>
> *Chuỗi đơn vị tương tự nhưng đơn giản hơn — chỉ có hai mốc, không có nghiên cứu khoa học, và Bằng khen Thủ tướng cấp đơn vị có thể nhận* ***lặp lại*** *sau mỗi bảy năm, khác với cá nhân là một lần duy nhất.*

[Chuyển]: *Sau khi đã nắm được nghiệp vụ, em xin chuyển sang phần thiết kế hệ thống.*

Slide 7 — Kiến trúc tổng quan (55 giây)

[Mở slide]: Sơ đồ Browser → Frontend → Backend → PostgreSQL + Socket.IO + PM2.

[Lời văn]:

> *Hệ thống chia làm bốn tầng. Trình duyệt giao tiếp với Frontend Next.js qua HTTP. Frontend gọi REST API tới Backend Express, đồng thời mở kết nối WebSocket Socket.IO để nhận thông báo thời gian thực. Backend dùng Prisma ORM truy cập PostgreSQL. Toàn bộ Backend và Frontend chạy dưới PM2 để tự khởi động lại khi crash, có log rotation và giới hạn bộ nhớ.*
>
> *Em chọn kiến trúc này vì ba lý do: tách biệt rõ Frontend và Backend giúp test riêng được; Socket.IO cho phép push thông báo mà không cần polling; và PM2 cho việc triển khai trên mạng nội bộ trở nên đơn giản, chỉ cần một lệnh.*

[Chuyển]: *Tiếp theo là chi tiết về stack công nghệ.*

Slide 8 — Tech Stack (50 giây)

[Mở slide]: 2 cột — Frontend / Backend + 6 pill công cụ phụ trợ.

[Lời văn]:

> *Phía Frontend, em dùng Next.js 14 với App Router để routing theo file system. Component library là Ant Design — em chọn vì có sẵn các component nghiệp vụ phức tạp như Table phân trang, Form validation chuẩn, DatePicker tiếng Việt. Tailwind CSS và shadcn/ui dùng cho phần style tuỳ biến. Validate dùng Zod, schema được chia sẻ giữa client và server để đảm bảo nhất quán.*
>
> *Phía Backend, em dùng Express với TypeScript, ORM là Prisma 6.x. Em chọn Prisma vì nó sinh ra TypeScript Client thực tế trong build, có autocomplete và compile-time check toàn diện — an toàn hơn các ORM dùng decorator. Database PostgreSQL hỗ trợ transaction SERIALIZABLE cần thiết cho test race condition. Xác thực dùng JWT hai token — access token ngắn để giảm rủi ro nếu lộ, refresh token dài để người dùng không phải đăng nhập lại liên tục.*
>
> *Các thư viện phụ trợ gồm Socket.IO cho real-time, PM2 quản lý process, node-cron lập lịch backup, ExcelJS cho import export, và PDFKit để sinh quyết định PDF.*

[Chuyển]: *Em xin trình bày thiết kế cơ sở dữ liệu.*

Slide 9 — Thiết kế CSDL (50 giây)

[Mở slide]: ERD bên trái + tổng quan 23 bảng bên phải.

[Lời văn]:

> *Cơ sở dữ liệu có 23 bảng chính, ID dạng CUID 25 ký tự — sortable theo thời gian mà không lộ thứ tự, an toàn hơn dùng auto-increment. Em chia 23 bảng thành bốn nhóm chức năng: nhóm tổ chức gồm Quân nhân, Cơ quan đơn vị, Đơn vị trực thuộc, Chức vụ; nhóm đề xuất và hồ sơ gồm Bảng đề xuất và các bảng hồ sơ cho từng loại khen thưởng; nhóm tài khoản với bốn vai trò; và nhóm phụ trợ gồm Audit log, Thông báo, File quyết định và Cấu hình hệ thống.*
>
> *Một điểm thiết kế đặc biệt em muốn nhấn mạnh: bảng `FileQuyetDinh` được liên kết với tám bảng khen thưởng qua **hard FK trên natural key `so_quyet_dinh`** thay vì surrogate ID. Điều này cho phép Postgres tự cascade rename khi cán bộ đổi số quyết định, đảm bảo tính toàn vẹn ở tầng database.*

[Chuyển]: *Tiếp theo là phân tích chức năng theo Use-case.*

Slide 10 — Use-case Diagram (50 giây)

[Mở slide]: Bảng 4 vai trò bên trái + sơ đồ use-case bên phải.

[Lời văn]:

> *Hệ thống có bốn vai trò phân theo cây phân cấp. USER là người dùng thông thường — chỉ xem hồ sơ cá nhân và nhận thông báo. MANAGER là chỉ huy đơn vị — quản lý dữ liệu trong phạm vi đơn vị mình phụ trách. ADMIN là cán bộ Phòng Chính trị — xử lý toàn bộ nghiệp vụ khen thưởng và phê duyệt đề xuất. SUPER_ADMIN là bộ phận kỹ thuật — quản trị hệ thống, backup, xem audit log toàn diện.*
>
> *Em hiện thực **hai lớp phân quyền** đồng thời: middleware `requireRole` chặn ở route — quyết định ai gọi được endpoint nào; và middleware `unitFilter` giới hạn dữ liệu trả về theo phạm vi đơn vị — đảm bảo MANAGER không thấy được dữ liệu đơn vị khác. Hai lớp này độc lập nên dù bypass được lớp này thì lớp kia vẫn chặn.*

[Chuyển]: *Em sẽ trình bày một quy trình nghiệp vụ tiêu biểu.*

Slide 11 — Activity Diagram quy trình duyệt (60 giây)

[Mở slide]: Activity diagram bên trái + 5 step bên phải.

[Lời văn]:

> *Quy trình duyệt đề xuất đi qua ba cấp. Bước một, Chỉ huy đơn vị tạo đề xuất, có thể nhập tay từng người hoặc import hàng loạt từ Excel. Bước hai, đề xuất chuyển đến Phòng Chính trị để duyệt — ở bước này hệ thống chạy bốn lớp kiểm tra trước khi cho phép phê duyệt: kiểm tra trạng thái đề xuất, kiểm tra trùng lặp với khen thưởng đã có, kiểm tra điều kiện chuỗi và niên hạn cống hiến, kiểm tra hợp lệ số quyết định.*
>
> *Sau khi qua bốn lớp này, bước ba đến năm chạy trong **một transaction Prisma duy nhất**: tạo bản ghi File quyết định, lưu khen thưởng vào hồ sơ quân nhân, cập nhật trạng thái đề xuất với khoá lạc quan để chống race condition. Nếu bất kỳ bước nào fail, toàn bộ rollback. Sau khi commit, hệ thống tính lại hồ sơ liên quan, ghi audit log và phát thông báo real-time đến chỉ huy đơn vị và quân nhân được nhận khen thưởng.*

[Chuyển]: *Em xin trình bày một điểm thiết kế chuyên sâu — Strategy pattern.*

Slide 12 — Strategy Pattern (60 giây) ⭐⭐

[Mở slide]: 2 cột — Vấn đề + giải pháp (trái) + Class diagram (phải).

[Lời văn]:

> *[nhìn slide] Bài toán đặt ra như sau: năm nhóm khen thưởng UC5 đến UC9 được hiện thực thành **bảy loại đề xuất** ở backend, mỗi loại có logic submit, validate và import vào database **khác nhau**. [ngừng] Nếu em viết theo `if/else` thuần, file điều phối sẽ lên đến **khoảng hai nghìn dòng** — rất khó test riêng từng nhánh và khó bảo trì.*
>
> *Em đã refactor sang **Strategy pattern**. Em định nghĩa một interface chung tên `ProposalStrategy` với **bốn phương thức**: một là tạo payload lúc submit, hai là validate lúc duyệt, ba là ghi vào database trong một transaction, và bốn là tạo thông báo thành công. [ngừng] Mỗi loại đề xuất có một file strategy riêng implement đầy đủ bốn phương thức này. Khi có request mới, service tra **Registry** để lấy đúng strategy theo loại đề xuất, rồi gọi phương thức tương ứng — service không cần biết chi tiết của loại nào. [ngừng]*
>
> *[nhấn] Một điểm em muốn nhấn mạnh — đây cũng là một quyết định thiết kế em đã cân nhắc kỹ. Hai loại Huân chương Quân kỳ Quyết thắng và Kỷ niệm chương có flow nhập liệu giống nhau đến **khoảng chín mươi phần trăm** — đều là khen thưởng cấp một lần cho một quân nhân, cùng cách validate ngày nhận và tính thời gian phục vụ. Em rút phần chung vào helper `singleMedalImporter` và inject phần khác — tức là bảng đích — qua một callback. Nhờ vậy code DRY mà không bloat interface chung. [ngừng]*
>
> *Lợi ích cuối cùng của thiết kế này: nếu sau này luật bổ sung loại khen thưởng mới, em chỉ cần **tạo một file strategy mới và đăng ký vào Registry** — không phải sửa code điều phối ở tầng controller hay service.*

[Chuyển]: *Em xin chuyển sang phần ba — bốn nhóm tính năng nổi bật của hệ thống.*

Slide 13 — [Feature 1] Tự động xét điều kiện (55 giây)

[Mở slide]: 4 feature card bên trái + screenshot timeline bên phải.

[Lời văn]:

> *Tính năng nổi bật thứ nhất là tự động xét điều kiện khen thưởng. Em hiện thực **một helper lõi tên `chainEligibility`** dùng chung cho cả hai chiều: chiều một là khi tính lại hồ sơ định kỳ thì sinh ra cờ "đủ điều kiện" và "gợi ý đề xuất"; chiều hai là khi cán bộ submit đề xuất, server validate lại bằng đúng helper đó. Hai chiều cùng một logic giúp đảm bảo những gì hệ thống hiển thị "đủ điều kiện" thì cán bộ submit chắc chắn pass.*
>
> *Hệ thống cũng tự sinh gợi ý dạng văn bản. Ví dụ trong hồ sơ năm 2026 của một quân nhân, hệ thống ghi: "Đủ điều kiện đề nghị Bằng khen Bộ Quốc phòng năm 2026" — cán bộ chỉ cần xác nhận để tạo đề xuất, không phải tự kiểm tra.*
>
> *Mỗi khi có thay đổi — quân nhân nhận danh hiệu mới, đổi đơn vị, cập nhật năm phục vụ — hệ thống tự recalc toàn bộ chuỗi của quân nhân đó. Nhờ vậy hồ sơ luôn đồng bộ với dữ liệu thực tế.*

[Chuyển]: *Tính năng thứ hai gắn với quy trình ra quyết định khen thưởng.*

Slide 14 — [Feature 2] Quy trình đề xuất + ra quyết định (55 giây)

[Mở slide]: 4 feature card bên trái + screenshot trang duyệt bên phải.

[Lời văn]:

> *Tính năng thứ hai là quy trình đề xuất và ra quyết định được số hoá toàn diện. Mỗi nhóm khen thưởng UC5 đến UC9 có form đề xuất riêng với các trường nghiệp vụ phù hợp. Schema validate dùng Zod,* ***chia sẻ giữa client và server*** *— viết một lần, dùng hai chỗ, đảm bảo không lệch quy tắc.*
>
> *Workflow đi qua ba cấp duyệt như em đã trình bày. Khi đề xuất được phê duyệt, hệ thống tự cấp số quyết định theo định dạng và bộ đếm riêng cho từng năm, đảm bảo không trùng lặp. Sau đó hệ thống áp dụng template theo từng loại danh hiệu để sinh PDF quyết định, lưu vào thư mục storage tập trung và đính kèm vào đề xuất.*
>
> *Toàn bộ quy trình thực hiện trong một transaction duy nhất ở tầng database, kèm audit log chi tiết và phát thông báo real-time cho người liên quan.*

[Chuyển]: *Tính năng thứ ba phục vụ nhập dữ liệu hàng loạt.*

Slide 15 — [Feature 3] Import / Export Excel (50 giây)

[Mở slide]: 4 feature card bên trái + screenshot preview Excel bên phải.

[Lời văn]:

> *Tính năng thứ ba là nhập xuất Excel hàng loạt theo pattern hai bước an toàn. Bước một là Preview — hệ thống parse file Excel, validate từng dòng theo định dạng ngày, CCCD, mã quân nhân, mã đơn vị. Cán bộ thấy trên giao diện một bảng có cột trạng thái Hợp lệ hoặc Lỗi, kèm thông báo lỗi cụ thể từng dòng. Bước hai là Confirm — sau khi cán bộ rà soát và xác nhận, hệ thống ghi vào database trong* ***một transaction****. Nếu một dòng fail, toàn bộ rollback, đảm bảo không có dữ liệu nửa vời.*
>
> *Template Excel được sinh tự động với data validation sẵn có — dropdown cho cấp bậc, đơn vị, danh hiệu. Cán bộ điền theo mẫu nên giảm sai sót đầu vào.*
>
> *Về hiệu năng, em áp dụng kỹ thuật batch query — collect các ID cần kiểm tra rồi gọi* `findMany` *một lần, đẩy vào Map để tra cứu O(1). Nhờ vậy thay vì N+1 truy vấn cho mỗi dòng dữ liệu, hệ thống chỉ cần một truy vấn cho toàn bộ batch — giảm đáng kể số lượt round-trip database.*

[Chuyển]: *Tính năng thứ tư về vận hành hệ thống.*

Slide 16 — [Feature 4] Vận hành & Quản trị (50 giây)

[Mở slide]: 3 cột — Notification / Audit / Backup.

[Lời văn]:

> *Tính năng thứ tư phục vụ vận hành. Về thông báo, mỗi sự kiện liên quan như duyệt đề xuất, từ chối, ra quyết định đều phát Socket.IO theo room user_id của người nhận. Đồng thời, thông báo cũng được lưu vào bảng Thông báo trong database, để nếu người dùng đang offline thì khi đăng nhập lại vẫn xem được.*
>
> *Về audit log, em hiện thực một middleware tự động ghi nhận mọi thao tác có ý nghĩa nghiệp vụ — ai làm gì, trên đối tượng nào, kèm IP và user-agent. Cán bộ có thể lọc theo người thực hiện, theo loại tài nguyên, theo khoảng thời gian. Một điểm phân quyền đặc biệt: log liên quan đến backup chỉ SUPER_ADMIN xem được, ADMIN và MANAGER bị filter loại bỏ.*
>
> *Về backup, em lập lịch chạy node-cron trong cùng process Express, kết xuất file SQL vào thư mục backups. SUPER_ADMIN có thể bật, tắt schedule, trigger thủ công, hoặc cleanup file cũ qua DevZone.*

> *Restore thực hiện bằng lệnh `psql` trên server.*

[Chuyển]: *Em xin trình bày phần kiểm thử và triển khai.*

Slide 17 — Kiểm thử + Hiệu năng + Triển khai (65 giây) ⭐

[Mở slide]: 2 cột — Test/hiệu năng (trái) + Triển khai (phải).

[Lời văn]:

> *Về kiểm thử, em viết **bảy mươi tư file test** với Jest, pass **một trăm phần trăm**. Em xin nhấn mạnh **bốn kịch bản tiêu biểu** trong các bộ test em viết.*
>
> *[nhìn slide] Thứ nhất là **race condition** — em mô phỏng hai cán bộ Phòng Chính trị duyệt song song cùng một đề xuất. Nhờ transaction kết hợp khoá lạc quan, chỉ một request thắng, request còn lại bị từ chối với thông báo rõ ràng. [ngừng]*
>
> *Thứ hai là **kịch bản chuỗi danh hiệu** với cửa sổ trượt bảy năm và lifetime — đảm bảo hệ thống **chặn đúng** khi quân nhân đã nhận Bằng khen Thủ tướng, và **mở đúng** khi đến chu kỳ kế. [ngừng]*
>
> *Thứ ba là **chống tampering** — em mô phỏng cán bộ cố sửa payload bypass giao diện để chèn dữ liệu sai quy định. Vì server validate lại đầy đủ ở tầng nghiệp vụ, dữ liệu vẫn được ghi đúng. [ngừng]*
>
> *Thứ tư là **kịch bản end-to-end** — mô phỏng vòng đời một quân nhân qua nhiều năm với việc lên cấp danh hiệu, lỡ đợt và thưởng đột xuất xen kẽ — kiểm tra tổng thể tính nhất quán của hệ thống. [ngừng]*
>
> *Về **hiệu năng**, như đã đề cập, em áp dụng kỹ thuật **batch query** trong import Excel — gộp truy vấn database thay vì gọi lặp từng dòng — đảm bảo hệ thống xử lý mượt với danh sách lớn.*
>
> *Về **triển khai**, hệ thống vận hành hoàn toàn trong mạng nội bộ. Quy trình cài đặt chỉ gồm bốn bước, kết thúc bằng lệnh `npm run serve` — PM2 sẽ tự build và khởi chạy với cấu hình auto-restart, giới hạn bộ nhớ năm trăm megabyte và log rotation.*

[Chuyển]: *Em xin chuyển sang phần kết luận và hướng phát triển.*

Slide 18 — Kết luận + Hướng phát triển (55 giây)

[Mở slide]: 3 cột — Đã đạt / Hạn chế / Hướng phát triển.

[Lời văn]:

> *[nhìn slide] Để **tổng kết**, đồ án của em đã hoàn thành các mục tiêu đặt ra ban đầu: hỗ trợ đầy đủ **năm nhóm khen thưởng** UC5 đến UC9, **bốn vai trò** phân quyền, có **bảy mươi tư** file kiểm thử bao phủ các kịch bản khó, và đã triển khai chạy được trên mạng nội bộ. Tổng dự án khoảng **một trăm nghìn** dòng code với kiến trúc layered và Strategy pattern rõ ràng. [ngừng]*
>
> *Em cũng nhận thấy **một số hạn chế** cần thẳng thắn đề cập. Thứ nhất, Bằng khen Thủ tướng cá nhân đang là lifetime — hệ thống chưa hỗ trợ danh hiệu cao hơn vì luật hiện hành chưa có quy định. Thứ hai, chưa có module thống kê dạng BI chi tiết, mới chỉ có dashboard cơ bản. Thứ ba, chỉ chạy một instance PM2 — chưa scale ngang được. Thứ tư, việc ký quyết định mới ở mức lưu file PDF, chưa tích hợp ký số bằng Smart Card. [ngừng]*
>
> *Trên cơ sở đó, **hướng phát triển tiếp theo** của em gồm bốn nội dung: xây dựng ứng dụng di động cho cán bộ duyệt nhanh; phát triển module thống kê BI với biểu đồ xu hướng theo năm và đơn vị; tích hợp ký*

> *số bằng Smart Card hoặc USB token; và mở rộng sang cluster mode với Redis adapter cho Socket.IO khi quy mô tăng.*

[Chuyển]: *Phần trình bày của em đến đây là hết. Em xin sang slide cuối.*

Slide 19 — Cảm ơn (20 giây)

[Mở slide]: Lời cảm ơn.

[Lời văn]:

> *Một lần nữa, em xin gửi lời cảm ơn sâu sắc đến **[Học hàm. Tên giảng viên]** — người đã tận tình hướng dẫn em trong suốt quá trình thực hiện đồ án. [ngừng]*
>
> *Em xin chân thành cảm ơn các thầy, cô trong Hội đồng đã lắng nghe phần trình bày. Em rất mong nhận được nhận xét, đánh giá và câu hỏi từ Hội đồng để hoàn thiện đồ án.*
>
> *[nhấn] **Em xin trân trọng cảm ơn ạ.***
>
> *(Cúi chào nhẹ, đứng tại chỗ chờ Hội đồng phản hồi — không vội về ghế ngồi)*

Phụ lục A — Kịch bản vào phần Q&A

> *Đây là phần **dễ mất điểm** nhất nếu không chuẩn bị. Học thuộc các câu mở/đóng phản hồi.*

A.1 — Khi thầy/cô bắt đầu hỏi

Tư thế: Đứng thẳng tại bục, hai tay trước, **nhìn thẳng vào người hỏi** trong suốt câu hỏi. Có thể ghi nhanh từ khoá ra giấy. Không cắt lời.

Câu mở phản hồi (chọn 1, dùng nhất quán):

> *"Em xin cảm ơn câu hỏi của thầy/cô. Em xin phép được trả lời như sau ạ..."*

> *"Dạ, em cảm ơn thầy/cô đã đặt câu hỏi. Về [tóm tắt 1 cụm từ trong câu hỏi], em xin phép trả lời ạ..."*

→ Tác dụng: **mua thêm 2-3 giây** để sắp xếp câu trả lời, đồng thời thể hiện sự lễ phép.

A.2 — Khi nghe câu hỏi mà chưa rõ ý

> *"Em xin phép được hỏi lại để làm rõ — ý của thầy/cô là [diễn giải lại ngắn gọn] ạ?"*

→ **Không đoán bừa**. Hỏi lại là OK, an toàn hơn trả lời sai.

A.3 — Khi không biết câu trả lời

Tuyệt đối không bịa. Có 3 cách xử lý theo mức độ:

1. **Biết một phần**: "Phần [X] em đã hiện thực như sau... [trình bày phần em biết]. Còn phần [Y] em **chưa nghiên cứu sâu**, em xin phép ghi nhận để bổ sung sau ạ."
2. **Biết hướng nhưng chưa làm**: "Hệ thống của em hiện chưa có tính năng đó. Theo em hiểu, nếu phát triển sẽ làm theo hướng [...]. Em xin nhận góp ý của thầy/cô ạ."

3. **Hoàn toàn không biết**: "Câu hỏi này em **chưa có đủ kiến thức** để trả lời. Em xin phép ghi nhận để nghiên cứu thêm sau buổi bảo vệ ạ."

→ Câu **3 chỉ dùng khi thực sự bí**. Hội đồng đánh giá cao sự thẳng thắn, không phải sự bịa.

A.4 — Khi thầy/cô không hỏi mà nhận xét

Nhiều thầy/cô không hỏi mà **góp ý**. Phản hồi:

> *"Em xin cảm ơn nhận xét của thầy/cô. Em sẽ tiếp thu và bổ sung ạ."*

→ **Không tranh luận**. Kể cả khi nhận xét chưa chính xác — buổi bảo vệ không phải lúc tranh luận.

A.5 — Khi thầy/cô ngắt giữa chừng để hỏi

> *"Dạ, em xin phép được trả lời câu hỏi của thầy/cô trước. [Trả lời]. Em xin phép tiếp tục phần đang trình bày ạ."*

→ Không bối rối. Trả lời rồi quay lại đúng slide đang dở.

A.6 — Câu kết thúc Q&A

Khi cảm thấy hội đồng không còn câu hỏi:

> *"Nếu Hội đồng không còn câu hỏi nào nữa, em xin phép được kết thúc phần trình bày tại đây. Em xin chân thành cảm ơn các thầy, cô ạ."*

(Cúi nhẹ, chờ chủ tịch hội đồng ra hiệu rồi mới về ghế ngồi.)

Phụ lục B — Ngân hàng câu hỏi Q&A

> *Đọc kỹ trước khi bảo vệ. Trả lời bình tĩnh, nhìn thẳng vào người hỏi, không vòng vo. Mỗi câu trả lời tối đa **45 giây** — quá dài sẽ bị cảm giác "vòng vo che giấu".*

Nhóm 1 — Lựa chọn công nghệ

Q1. Tại sao em chọn Prisma mà không dùng TypeORM hay Sequelize?

> *Em chọn Prisma vì ba lý do. Một, Prisma sinh TypeScript Client thực tế trong build — autocomplete và type-check ở compile time toàn diện, an toàn hơn TypeORM dùng decorator + reflection chỉ check một chiều. Hai, Prisma có hệ thống migration declarative, viết schema rồi `prisma migrate dev` tự sinh SQL — không phải viết migration tay như Sequelize. Ba, Prisma Studio dùng để debug DB rất tiện trong quá trình phát triển.*

Q2. Tại sao Next.js mà không dùng React thuần với Vite?

> *Next.js có App Router routing theo file system rõ ràng — không cần thiết lập react-router thủ công. Server components giúp giảm payload JavaScript gửi xuống client. Khi đóng gói, `next build` sinh sẵn standalone bundle có thể chạy bằng `node` không cần dev server — phù hợp với deploy mạng nội bộ.*

Q3. Sao chọn JWT hai token thay vì session?

> *Session cần lưu state ở server, gây khó khi scale ngang. JWT stateless. Em chọn hai token: access TTL ngắn — mười lăm phút — để giảm rủi ro nếu lộ; refresh token TTL bảy ngày để người dùng không phải đăng nhập lại liên tục. Khi access hết hạn, FE gọi `/refresh` để xin token mới mà không cần login lại.*

Q4. Tại sao PostgreSQL chứ không dùng MySQL?

> *PostgreSQL hỗ trợ isolation level SERIALIZABLE đầy đủ — em cần cho test race condition. Nó cũng hỗ trợ JSONB column native — em dùng cho payload đề xuất linh hoạt theo loại. Ngoài ra, kiểu dữ liệu phong phú hơn MySQL như array, ENUM thực sự, và type safety tốt hơn.*

Nhóm 2 — Thiết kế kiến trúc

Q5. Tại sao 23 bảng nhiều như vậy, có chuẩn hoá không?

> *Em đã chuẩn hoá đến 3NF. 23 bảng vì có bảy loại khen thưởng, mỗi loại có hồ sơ và bảng khen thưởng riêng do **cấu trúc field khác nhau** — ví dụ Hồ sơ Niên hạn có ba cột status cho ba hạng HCCSVV, còn Hồ sơ Cống hiến có thêm các cột tổng tháng theo hệ số 0.7, 0.8, 0.9-1.0. Nếu gộp một bảng thì phải null nhiều cột không liên quan và khó query.*

Q6. Strategy pattern có over-engineering không?

> *Em từng viết theo `if/else` ban đầu và phát hiện file `approve.ts` lên hai nghìn dòng, khó test riêng từng nhánh. Refactor sang Strategy giúp mỗi loại độc lập, có thể test riêng, và thêm loại mới chỉ thêm một file. Trade-off chấp nhận được khi có bảy loại đề xuất khác biệt.*

Q7. Repository pattern có cần thiết không khi đã có Prisma?

> *Có. Repository decouple business logic khỏi Prisma — service không gọi `prisma.xxx` trực tiếp mà gọi `xxxRepository.findById`. Lợi ích: một, dễ mock khi unit test service; hai, nếu sau này đổi ORM (ví dụ sang Drizzle), chỉ sửa repository, service không thay đổi; ba, có thể inject `tx` (transaction client) thống nhất qua tham số repository.*

Q8. Tại sao tách Frontend và Backend mà không dùng Next.js full-stack?

> *Em có cân nhắc. Tách ra giúp Backend test riêng được bằng Jest + Supertest, không cần spawn Next.js. Backend cũng có thể serve cho nhiều client khác trong tương lai như mobile app. Trade-off là phải config CORS và quản lý hai process — em dùng PM2 nên không phải vấn đề lớn.*

Nhóm 3 — Bảo mật

Q9. Phòng XSS, SQL injection thế nào?

> *SQL injection: Prisma parameterized query toàn bộ, em không dùng `$queryRawUnsafe` với input từ user. XSS: React tự escape khi render, em không dùng `dangerouslySetInnerHTML`. Validate input qua Zod ở client và Joi ở server — reject bất kỳ field không có trong schema bằng `stripUnknown: true`.*

Q10. Race condition em xử lý thế nào?

> *Em dùng hai cơ chế. Thứ nhất là Prisma transaction `prisma.$transaction()` cho các luồng cần atomic. Thứ hai là **optimistic lock** — `updateMany` với điều kiện `status: 'PENDING'`, nếu count trả về 0 thì có người đã update trước, em throw conflict error. Test với `Promise.all` hai request duyệt song song, verify chỉ một thắng.*

Q11. Tampering thì sao? Cán bộ có thể sửa payload bypass FE.

> *Em test cụ thể tình huống này. Mọi validation business critical đều chạy ở **server-side** trong service, không phụ thuộc FE. Ví dụ kiểm tra eligibility chuỗi, kiểm tra quyền theo role, kiểm tra phạm vi đơn vị — đều ở backend. FE chỉ làm UX validation cho phản hồi nhanh.*

Q12. Audit log có làm chậm hệ thống không?

> *Audit log ghi đồng bộ trong middleware sau controller. Một số log fire-and-forget dùng `void writeSystemLog(...)` — không await. Trong stress test, overhead khoảng năm đến mười mili-giây mỗi request, chấp nhận được vì yêu cầu nghiệp vụ bắt buộc.*

Nhóm 4 — Triển khai

Q13. Mạng nội bộ làm sao npm install?

> *Em làm theo hai cách. Một, cài `node_modules` một lần trên máy có Internet, copy sang server bằng USB hoặc rsync — chỉ phải làm khi đổi dependency. Hai, đặt local npm registry như Verdaccio làm proxy cache trong mạng nội bộ — `npm install` đầu tiên proxy ra ngoài, các lần sau dùng cache.*

Q14. Backup làm sao restore?

> *File backup là SQL plain text sinh từ `pg_dump` qua service em viết — gồm toàn bộ INSERT statements. Restore bằng lệnh `psql -d qlkt < backup.sql` trên server. Em chưa expose endpoint download qua HTTP để giảm rủi ro lộ dữ liệu — phải SSH lấy file thủ công.*

Q15. Nếu PM2 process chết trong khi đang import Excel thì sao?

> *Import Excel thực thi trong **một transaction Prisma**, nên nếu process chết giữa chừng, transaction tự rollback ở DB, không có dữ liệu nửa vời. Khi PM2 auto-restart, hệ thống quay về trạng thái nhất quán. Cán bộ chỉ cần upload file lại.*

Q16. Số liệu test 74 file có thật không?

> *Em chạy `npx jest --listTests | wc -l` trước khi báo cáo, kết quả đúng bảy mươi tư file. Em có thể demo trực tiếp trên máy nếu thầy/cô muốn xem.*

Nhóm 5 — Nghiệp vụ

Q17. BKTTCP lifetime nghĩa là gì?

> *BKTTCP — Bằng khen Thủ tướng Chính phủ cá nhân — chỉ được nhận một lần trong đời. Sau khi nhận, hệ thống chặn các đề xuất tiếp theo và hiển thị thông báo "Đã có Bằng khen Thủ tướng. Phần mềm chưa hỗ trợ các danh hiệu cao hơn Bằng khen Thủ tướng, sẽ phát triển trong thời gian tới." Vì luật hiện hành chưa quy định danh hiệu cao hơn cho cá nhân.*

Q18. Cửa sổ trượt là gì?

> *Khi xét điều kiện một danh hiệu cấp cao, hệ thống chỉ đếm các flag trong **N năm gần nhất** tính từ năm xét. Ví dụ xét Chiến sĩ Thi đua Toàn quốc năm 2026, hệ thống đếm Bằng khen Bộ Quốc phòng trong ba năm 2024-2026. Bằng khen của các năm trước rơi ra khỏi cửa sổ, không tính. Đây là cách luật quy định để đảm bảo "khen thưởng phải xứng đáng theo thành tích gần nhất".*

Q19. Lỡ đợt nghĩa là gì?

> *Lỡ đợt là khi đủ điều kiện đề nghị một danh hiệu nhưng đơn vị không đề xuất. Trong hệ thống của em, chu kỳ vẫn tiếp tục đếm — quân nhân không phải bắt đầu lại chuỗi Chiến sĩ Thi đua Cơ sở từ đầu. Đến*

> *chu kỳ kế tiếp (cộng số năm chu kỳ), nếu đáp ứng điều kiện, hệ thống lại gợi ý đề nghị. Đây là điểm em phải cài cẩn thận vì luật cho phép như vậy nhưng nhiều phần mềm tính sai.*

Q20. Phân biệt khen thưởng cá nhân và đơn vị thế nào?

> *Hai chuỗi riêng biệt, không liên quan. Cá nhân là cho từng quân nhân, dựa trên CSTDCS hằng năm và NCKH. Đơn vị là cho cả Cơ quan đơn vị hoặc Đơn vị trực thuộc, dựa trên Đơn vị Quyết thắng hằng năm — không cần NCKH. Bằng khen Thủ tướng cấp đơn vị có thể nhận lặp lại sau mỗi bảy năm, khác với cá nhân là lifetime.*

Nhóm 6 — Quá trình thực hiện đồ án

Q21. Khó khăn lớn nhất khi làm đồ án là gì?

> *Khó khăn lớn nhất là **hiểu đúng nghiệp vụ** chuỗi danh hiệu. Em đã đọc nhiều văn bản pháp lý — Luật Thi đua Khen thưởng, Nghị định 98 và các thông tư liên quan — và phỏng vấn cán bộ Phòng Chính trị nhiều lần để xác nhận cách hiểu. Có những trường hợp em hiểu sai cửa sổ trượt và lỡ đợt, phải sửa lại logic core nhiều lần. Đây là lý do em cài hai lớp validate đồng bộ — để nếu hiểu sai chỗ nào thì sai cả hai chỗ chứ không bị lệch giữa hiển thị và xét duyệt.*

Q22. Em đã khảo sát người dùng thực tế thế nào?

> *Em đã làm việc với cán bộ Phòng Chính trị tại đơn vị em được giới thiệu. Em xem trực tiếp file Excel và biểu mẫu giấy họ đang dùng, ghi nhận các trường dữ liệu thực tế và quy trình duyệt. Form đề xuất trong phần mềm em thiết kế có cùng các trường với biểu mẫu giấy để cán bộ chuyển đổi không bỡ ngỡ.*

Q23. Em làm đồ án bao lâu?

> *Em bắt đầu khảo sát và thiết kế từ [tháng/năm], hiện thực và kiểm thử trong [N] tháng. Tổng cộng khoảng [N] tháng. Phần khó nhất tốn nhiều thời gian là hiểu nghiệp vụ và refactor sang Strategy pattern.*

Nhóm 7 — Câu hỏi mở rộng

Q24. Tại sao không dùng GraphQL?

> *GraphQL hợp với UI có nhiều client (web, mobile, partner) và yêu cầu query field linh hoạt. Đồ án này chỉ có một client web nội bộ và endpoint khá ổn định. REST đơn giản, dễ debug bằng `curl`, có sẵn middleware ecosystem và documentation chuẩn cho các thầy cô review. Trade-off chấp nhận được.*

Q25. Có dùng CI/CD không?

> *Em chưa cài CI/CD chính thức vì hệ thống deploy nội bộ — không có pipeline tự động đến server đơn vị. Local em có script `npm run typecheck` và `npm run test` chạy thủ công trước commit. Nếu mở rộng, em sẽ thêm GitHub Actions chạy test khi push, và một workflow sinh artifact để cán bộ kỹ thuật pull về deploy.*

Q26. Hệ thống có scale được không nếu mở rộng cho nhiều đơn vị?

> *Hiện tại single instance PM2 đáp ứng tốt cho quy mô vài trăm người dùng đồng thời ở một đơn vị. Nếu mở rộng, em xác định ba điểm cần làm: một, thêm Redis adapter cho Socket.IO để scale ngang nhiều instance; hai, dùng PM2 cluster mode hoặc chuyển sang container; ba, tách database read replica cho các query thống kê nặng. Code đã sẵn sàng — em đã tách Repository layer nên đổi sang connection pool nhiều node không phải refactor business logic.*

Q27. Sao không dùng microservice?

> *Quy mô đồ án không yêu cầu microservice — overhead vận hành sẽ lớn hơn lợi ích. Em chọn **modular monolith**: kiến trúc layered tách rõ Controller, Service, Repository, Strategy — nếu sau này cần tách thành service riêng (ví dụ tách module Audit log), em chỉ cần đẩy một số thư mục `services/` ra service riêng và đổi gọi hàm thành gọi REST. Hiện tại deploy đơn giản hơn, debug dễ hơn.*

Q28. Documentation của hệ thống thế nào?

> *Em viết tài liệu ở ba tầng. Một, file `CLAUDE.md` ở root project là tài liệu kiến trúc và convention dùng cho việc bảo trì sau này — gồm naming, anti-pattern, rule chuỗi, sync map giữa code và báo cáo. Hai, JSDoc trên các hàm exported. Ba, thư mục `docs/diagrams/` có 8 file Mermaid cho 8 sơ đồ UML — use case, activity, class, sequence, ERD, deployment, architecture, đặc tả use case chi tiết. Sơ đồ render tự động trên VSCode, không cần tool ngoài.*

Q29. So với phần mềm thương mại đang có thì khác gì?

> *Em đã tham khảo một số phần mềm quản lý khen thưởng thương mại. Khác biệt chính là **logic chuỗi danh hiệu** — phần lớn phần mềm hiện có chỉ ghi danh hiệu theo năm, không tự xét chuỗi và gợi ý đề nghị. Cán bộ vẫn phải tra Excel thủ công. Đồ án của em tự động xét đầy đủ ba quy tắc cửa sổ trượt, lifetime, lỡ đợt — đây là điểm khác biệt cốt lõi.*

Q30. Nếu phải làm lại đồ án, em sẽ làm khác gì?

> *Em sẽ áp dụng Strategy pattern và Repository pattern **ngay từ đầu** thay vì refactor giữa chừng — sẽ tiết kiệm khoảng một tuần. Em cũng sẽ viết test sớm hơn cho phần chuỗi danh hiệu — em viết test khá muộn nên có những bug logic cửa sổ trượt phải sửa nhiều lần trước khi test phát hiện. Cuối cùng, em sẽ chuẩn bị môi trường demo trên một con server riêng từ sớm, không để khi gần bảo vệ mới setup.*

Phụ lục C — Cheat card (in 1 trang A4 cầm tay)

> *In phần này thành 1 trang A4, gập đôi đặt lên bục thuyết trình. Chỉ liếc khi cần. **Đừng đọc liên tục từ giấy.***

```
┌──────────────────────────────────────────────────────────┐
│ S1 (40s) Chào HĐ + cảm ơn GVHD + intro đề tài              │
│ S2 (20s) 4 phần: Mở đầu / Thiết kế / Tính năng / Đánh giá  │
│ S3 (55s) Excel thủ công → sai sót → mục tiêu 4 điểm        │
│ S4 (50s) 6 thách thức: nghiệp vụ + chuỗi + RBAC + RT + log  │
│ S5 (55s) UC5–UC9: hằng năm / niên hạn / cống hiến / NCKH /  │
│         đột xuất                                            │
│ S6 (90s)⭐ CHUỖI DANH HIỆU – 3 mốc + 3 quy tắc đặc biệt    │
│         BKBQP 2y → CSTDTQ 3y → BKTTCP 7y                    │
│         Cửa sổ trượt | Lifetime | Lỡ đợt                   │
│ S7 (55s) Browser → FE → BE → DB + Socket.IO + PM2           │
│ S8 (50s) Tech stack: Next.js + AntD + Express + Prisma      │
│ S9 (50s) 23 bảng / CUID / FK natural-key so_quyet_dinh      │
│ S10 (50s) 4 role + 2 lớp permission (route + unit)          │
│ S11 (60s) Activity: Manager → Admin, 4 lớp validate, TX     │
│ S12 (60s)⭐ STRATEGY PATTERN – 7 loại, interface 4 method    │
│         singleMedalImporter share HC_QKQT + KNC             │
│ S13 (55s) Tự động xét: 2-layer eligibility + goi_y + recalc│
│ S14 (55s) Form Zod share + 3 cấp duyệt + cấp số + PDF       │
│ S15 (50s) Excel: Preview → Confirm + batch query (no N+1)   │
```

```
│ S16 (50s) Notif Socket / Audit log / Backup cron + DevZone │
│ S17 (65s)⭐ 74 test pass 100% — race + chain + tampering +  │
│         e2e + deploy 4 bước                                 │
│ S18 (55s) Đạt: 5 nhóm + 4 role + 74 test + 100k LOC         │
│         Hạn chế: lifetime + BI + scale + ký số              │
│         HP: mobile + BI + Smart Card + cluster              │
│ S19 (20s) Cảm ơn GVHD + Hội đồng                            │
│                                                             │
│ TOTAL ~13.5 phút                                            │
└─────────────────────────────────────────────────────────────┘
```

Số liệu phải nhớ chính xác:

- 23 bảng — CSDL
- 74 file test — pass 100%
- 7 loại đề xuất — Strategy
- ~100k LOC — tổng dự án
- 4 vai trò: SUPER_ADMIN > ADMIN > MANAGER > USER
- 5 nhóm UC5-UC9
- Cửa sổ trượt: 3 năm cho CSTDTQ, 7 năm cho BKTTCP
- Import Excel: tối ưu **batch query** (1 `findMany` thay N+1)

Phụ lục D — Tình huống đặc biệt

D.1 — Mất kết nối / lỗi máy chiếu giữa chừng

> *"Em xin phép Hội đồng đợi một lát để em khắc phục kỹ thuật ạ."*

→ Bình tĩnh, không lúng túng. Nếu không khắc phục được trong 30s:

> *"Em xin phép tiếp tục trình bày bằng máy của em ạ" (mở laptop hướng về phía hội đồng) hoặc "Em xin trình bày chay phần còn lại, các sơ đồ em sẽ giải thích bằng lời và thầy/cô có thể xem chi tiết trong báo cáo ạ".*

D.2 — Demo fail trước hội đồng

> *"Có vẻ có lỗi nhỏ ở môi trường demo. Em xin phép trình bày bằng các screenshot trong slide ạ" → chuyển sang slide có screenshot, mô tả như demo thật.*

→ **Tuyệt đối không** debug live trước hội đồng. Lùi về slide.

D.3 — Hội đồng yêu cầu xem code

> *"Dạ, em xin phép mở source code ạ" → chuẩn bị sẵn IDE mở project trên màn hình thứ hai. Mở file/hàm mà câu hỏi liên quan, không scroll lung tung.*

→ Đảm bảo đã commit hết, không có TODO/FIXME nhạy cảm trong file đang mở.

D.4 — Hội đồng yêu cầu chứng minh số liệu

> *"Em xin phép chạy lệnh để Hội đồng xác nhận ạ" → mở terminal, chạy `npx jest --listTests | wc -l` (cho 74 test) hoặc `git ls-files | xargs wc -l` (cho LOC).*

→ Chuẩn bị trước các lệnh kiểm tra số liệu trong file `verify-stats.sh` hoặc memo riêng.

D.5 — Hội đồng đặt câu hỏi nằm ngoài phạm vi đồ án

> *"Câu hỏi của thầy/cô nằm ngoài phạm vi đồ án em đã thực hiện. Theo quan điểm cá nhân em hiểu là [...]. Tuy nhiên đây không phải nội dung em đã cài đặt và kiểm thử trong đồ án ạ."*

→ Tách rõ "đồ án em đã làm" với "ý kiến cá nhân" — không gộp lẫn.

D.6 — Hội đồng cho rằng số liệu chưa đáng tin

> *"Em xin phép trình bày cách em đo. [Tả lại phương pháp đo ngắn gọn.] Em sẵn sàng demo lại nếu Hội đồng yêu cầu ạ."*

→ Không tranh cãi. Trình bày phương pháp + offer demo.

D.7 — Hết giờ trình bày mà chưa xong slide

→ **Skip slide không quan trọng**. Theo thứ tự ưu tiên skip: Slide 8/9/10 (tech stack, ERD, use-case) → Slide 4 (thách thức) → Slide 16 (vận hành).

→ **Tuyệt đối không skip**: Slide 6 (chuỗi danh hiệu), Slide 12 (Strategy), Slide 17 (test+deploy), Slide 18 (kết luận).

Phụ lục E — Mẹo trình bày tổng hợp

E.1 — Trước buổi bảo vệ

1. **Đến sớm 15 phút** — kiểm tra máy chiếu, kết nối laptop, mở sẵn slide ở Presenter view, mở sẵn `localhost:3000` đã đăng nhập tài khoản ADMIN.
2. **Tổng duyệt** — tự bấm giờ đọc to kịch bản trước gương ít nhất **3 lần**, đảm bảo dưới 14 phút.
3. **Trang phục**: áo sơ-mi trắng + cà-vạt + quần tây đen (theo quy định HUST). Tóc gọn gàng.
4. **Chuẩn bị backup**: USB chứa file slide PDF + source code zipped, phòng trường hợp laptop hỏng.
5. **Ngủ đủ giấc** đêm trước, ăn nhẹ trước 30 phút (tránh đói hoặc no quá).
6. **Nước uống**: chai nước nhỏ đặt trên bục, uống một ngụm khi đổi slide nếu cần.

E.2 — Trong khi trình bày

7. **Đứng thẳng**, hai chân vững, **nhìn vào hội đồng** — không nhìn slide quá 5 giây liên tục. Slide chỉ là điểm tựa.
8. **Nhịp đọc**: 130–140 từ/phút. Đọc to + chậm + nhấn vào con số quan trọng và tên kỹ thuật.
9. **Câu chuyển slide**: luôn nói **trước khi click** — "Em xin chuyển sang...", "Tiếp theo...", "Em xin trình bày..." — tránh im lặng đột ngột.
10. **Đoạn khó nhất**: Slide 6 (chuỗi danh hiệu) và Slide 12 (Strategy). Tập đọc nhiều lần để trôi chảy. Đây là 2 slide chấm điểm cao nhất.
11. **Nếu vấp**: ngừng 1 giây, hít thở, không xin lỗi rườm rà — chỉ "Em xin nói lại ạ" rồi tiếp.
12. **Tay**: không đút túi quần, không khoanh tay trước ngực. Có thể cầm bút hoặc kẹp slide nhỏ.

E.3 — Trong phần Q&A

13. **Câu hỏi không biết trả lời**: thành thật "Em xin phép ghi nhận để nghiên cứu thêm" — **không bịa**.
14. **Câu trả lời tối đa 45 giây**. Vòng vo bị cảm giác "che giấu".
15. **Demo chuẩn bị sẵn**: nếu hội đồng yêu cầu, mở sẵn `localhost:3000` và một tài khoản ADMIN trước khi vào phòng.
16. **Không tranh luận** — kể cả khi nhận xét chưa chính xác. Cảm ơn rồi tiếp thu.
17. **Ghi chép**: chuẩn bị sổ tay nhỏ + bút, ghi nhanh từ khoá khi thầy/cô góp ý dài.

E.4 — Sau buổi bảo vệ

18. **Cúi chào Hội đồng** trước khi rời bục, không vội về ghế ngay khi nói "cảm ơn" cuối.
19. **Cảm ơn GVHD riêng** sau buổi bảo vệ — qua tin nhắn hoặc gặp trực tiếp.
20. **Lưu lại nhận xét** của hội đồng để sửa báo cáo bản cuối nếu được yêu cầu.
21. **Trang phục**: áo sơ-mi trắng + cà-vạt + quần tây đen (theo quy định HUST).
22. **Đến sớm 15 phút**: kiểm tra máy chiếu, kết nối laptop, mở sẵn slide ở Presenter view.
23. **Tổng duyệt**: tự bấm giờ đọc kịch bản trước gương ít nhất 3 lần — đảm bảo dưới 14 phút.